

Lab 1: Mini UNIX Shell

Technical Report

Nizaul Rahman (MCS252099)

December 8, 2025

1 Introduction

This report describes the design and implementation of `nizshell`, a simple UNIX-style shell written in C. The shell can run external programs, support quoted arguments, I/O redirection, pipelines, background execution, and built-in commands. Non-functional requirements such as safe input handling, zombie cleanup, and `Ctrl+C` protection are also implemented. Additionally, history and multi-stage pipelines were included as optional enhancements.

2 Requirements Summary

Functional Requirements

- **External commands:** Executed using `execvp()`, which performs PATH lookup.
- **Argument parsing:** Tokenizer handles normal words and both single/double quoted strings.
- **I/O redirection:** Supported through `<` and `>` operators and implemented via `dup2()`.
- **Pipelines:** Multi-stage pipelines implemented using an array of Command structures.
- **Background jobs:** `&` sets a background flag; parent skips `waitpid()`.
- **Built-ins:** `cd`, `exit`, and `history`.

Non-Functional Requirements

- Input validation prevents crashes on malformed commands.
- `SIGCHLD` handler cleans all zombie processes.
- Shell ignores `SIGINT`; children restore default behavior.
- Compiles cleanly using `gcc -Wall -Wextra`.

3 System Design

Overall Flow

The shell follows the classic read–parse–execute loop:

1. Print prompt and read input.
2. Store in history.
3. Tokenize into words and symbols.

4. Parse into one or more commands.
5. Execute a single command or pipeline.

Data Structures

Command representation:

```
typedef struct {  
    char *argv[64];  
    char *infile;  
    char *outfile;  
    int argc;  
} Command;
```

History buffer: fixed-size circular array storing the last 100 commands.

4 Tokenization & Parsing

The tokenizer splits input into tokens while correctly handling:

- quoted strings ("..."/'...')
- redirection operators
- pipeline symbols
- background operator

The parser builds an array of commands (up to 16 stages). Each stage can contain arguments and optional redirections. Error cases such as empty commands or missing filenames are handled gracefully.

5 Execution Model

Built-ins

Executed in the parent process:

- `cd`: uses `chdir()`.
- `exit`: frees history and terminates the shell.
- `history`: prints stored commands.

External Commands

A child process is forked:

1. Child resets `SIGINT`, applies redirection, then calls `execvp()`.
2. Parent waits unless background flag is set.

Pipelines

Multi-stage pipelines use:

- `pipe()` for communication
- `dup2()` to connect pipes to `stdin/stdout`
- multiple child processes (one per stage)

The parent closes all pipe descriptors and waits for pipeline completion unless running in background.

6 Signal Handling and Process Cleanup

- **SIGINT**: Shell ignores; children restore default so foreground jobs remain interruptible.
- **SIGCHLD**: Handler reaps all finished children using `waitpid(-1, WNOHANG)`.

This prevents zombie buildup and ensures clean operation even with pipelines and background jobs.

7 Testing Summary

The shell was tested using:

- Simple commands: `ls`, `pwd`
- Quoted strings: `echo "hello world"`
- Redirection: `wc < file`, `ls > out.txt`
- Pipelines: `ls | wc -l`, `cat x | grep y | wc -l`
- Background jobs: `sleep 5 &`
- Signal handling: `Ctrl+C` during long sleeps
- History: `history`

All required features behaved correctly.

8 Conclusion

The implemented `nizshell` satisfies all mandatory requirements of the lab and includes two optional features: multi-stage pipelines and command history. The project demonstrates understanding of UNIX processes, signals, pipes, and low-level command parsing. The shell is stable, does not crash on malformed input, and handles process cleanup correctly.